Comparación experimental de los algoritmos de ordenación por inserción y Shell sort en C++

Experimental comparison of the insertion and Shell sort algorithms in C++

Mikhael Marcos Ernesto Pacovilca Choque

mpacovilcac@unjbg.edu.pe

Escuela Profesional de Ingeniería en Informática y Sistemas, Facultad de Ingeniería

Universidad Nacional Jorge Basadre Grohmann

Tacna – Perú

Resumen:

Este trabajo comparó los algoritmos de ordenación por inserción y Shell sort en C++. Se midieron el tiempo de ejecución, las comparaciones y los intercambios con arreglos aleatorios, ascendentes y descendentes de 1 000, 10 000 y 100 000 elementos, con 30 repeticiones por caso. Con datos aleatorios y descendentes, el más rápido fue Shell sort; con datos ascendentes, fue inserción. Por ejemplo, con n = 100 000 y datos descendentes, inserción tardó 5 266 ms y Shell sort 2.3 ms. Se concluyó que inserción conviene cuando los datos ya están ordenados, y que Shell sort conviene con datos aleatorios o en orden descendente.

Palabras clave:

algoritmos de ordenación, complejidad computacional, inserción, Shell sort

Abstract:

This study compared the insertion sort and Shell sort algorithms in C++. Execution time, comparisons and swaps were measured on random, ascending and descending arrays of 1,000, 10,000 and 100,000 elements, with 30 repetitions per case. With random and descending data, Shell sort was the fastest; with ascending data, insertion sort was. For example, with n = 100,000 and descending data, insertion sort took 5,266 ms and Shell sort 2.3 ms. It was concluded that insertion sort is preferable when the data are already ordered, and that Shell sort is preferable with random or descending data.

Keywords: *sorting algorithms, computational complexity, insertion sort, Shell sort.*

1 Introducción

Ordenar datos es una tarea básica en programación y existen muchos algoritmos para hacerlo. Estos algoritmos no rinden igual con todos los datos: el tiempo que tardan depende de cómo vienen ordenados y de cuántos son.

Salgado et al. (2022) compararon el tiempo de ordenación de cinco algoritmos, entre ellos inserción, con 1 000, 10 000 y 100 000 datos, y encontraron diferencias significativas, siendo quicksort el más rápido. A diferencia de ese estudio, este trabajo compara inserción con Shell sort, además del tiempo mide comparaciones e intercambios, y analiza tres patrones de datos (aleatorio, ascendente y descendente).

El objetivo de este trabajo fue comparar los algoritmos de ordenación por inserción y Shell sort en C++, midiendo su tiempo de

ejecución, comparaciones e intercambios. Se esperaba que Shell sort fuera más rápido con datos aleatorios.

En la sección 2 se describe el método, en la 3 se presentan los resultados, en la 4 se discuten y en la 5 se dan las conclusiones.

2 Método

2.1 Algoritmos y variantes

La ordenación por inserción recorre el arreglo y coloca cada elemento en su lugar entre los de su izquierda, igual que se ordena una baraja de cartas. Shell sort aplica la misma idea, pero primero compara elementos separados por un salto (gap) grande y lo va reduciendo hasta llegar a 1. En este trabajo los saltos fueron n/2, n/4, …, 1. En el mejor caso, que ocurre cuando los datos ya están ordenados, inserción tiene complejidad O(n). En el peor caso, que ocurre cuando los datos están en orden descendente, tiene complejidad $O(n^2)$. Shell sort tiene el mismo peor caso, $O(n^2)$, pero en el mejor caso su complejidad queda entre O(n) y $O(n^2)$ (aproximadamente O(n log n)). Inserción es estable, porque solo mueve un elemento cuando es estrictamente menor y los elementos iguales conservan su orden. Shell sort no es estable, porque los saltos grandes pueden pasar un elemento por encima de otro igual.

2.2 Entorno experimental

Todas las pruebas se realizaron en el mismo equipo, con procesador Intel Core i5-12450H (8 núcleos, 12 hilos, 2.00 GHz base), 15.7 GB de RAM y Windows 11 Pro. Los programas se compilaron con g++ 16.2.0 (MSYS2 UCRT64) usando las opciones -O2 y -std=c++14.

2.3 Generación de datos

Se generaron tres patrones de datos: aleatorio, ascendente y descendente. En el patrón aleatorio se usaron números del 0 al 32 767 generados con rand(), con semilla 12345 más el número de repetición. Cada patrón se probó con tres tamaños: 1 000, 10 000 y 100 000 elementos.

2.4 Repeticiones y métricas

Se hicieron 30 repeticiones por cada combinación de algoritmo, patrón y tamaño, y en cada una ambos algoritmos ordenaron una copia del mismo arreglo. El tiempo se midió en milisegundos con std::chrono::steady_clock, sin contar lectura ni escritura, y además se contaron las comparaciones y los intercambios. Con las 30 repeticiones se calcularon el promedio, la mediana, la desviación estándar, el mínimo y el máximo.

3 Resultados

3.1 Tiempo de ejecución

La Tabla 1 muestra el tiempo de ejecución (ms) de ambos algoritmos para cada patrón (aleatorio, ascendente y descendente) y cada tamaño n.

Tabla 1: Tiempo de ejecución (ms), promedio ± desviación estándar de 30 repeticiones.

n	Patrón	Inserción	Shell sort
1000	aleatorio	0.1793 ± 0.0132	0.0491 ± 0.0078

n	Patrón	Inserción	Shell sort
1000	ascendente	0.0008 ± 0.0001	0.0069 ± 0.0005
1000	descendente	0.3745 ± 0.0516	0.0104 ± 0.0022
10000	aleatorio	25.7567 ± 6.3653	0.9726 ± 0.2075
10000	ascendente	0.0179 ± 0.0083	0.2266 ± 0.0938
10000	descendente	42.9449 ± 12.0692	0.1641 ± 0.0489
100000	aleatorio	2574.1180 ± 178.7124	13.6624 ± 2.4832
100000	ascendente	0.0954 ± 0.0025	1.4962 ± 0.1164
100000	descendente	5266.0327 ± 207.2387	2.3217 ± 0.4626

La Figura 1 muestra la distribución de los tiempos con n = 100 000. Con datos aleatorios y descendentes, inserción tardó mucho más que Shell sort; con datos ascendentes ocurrió lo contrario.

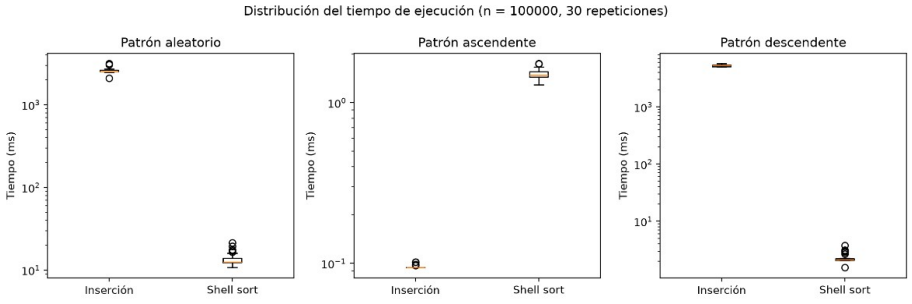


Figura 1: Distribución del tiempo de ejecución por algoritmo y patrón (n = 100 000, 30 repeticiones).

La Figura 2 muestra el tiempo promedio según n. Con datos aleatorios y descendentes, la curva de inserción crece mucho más rápido que la de Shell sort; con datos ascendentes, inserción fue la más rápida.

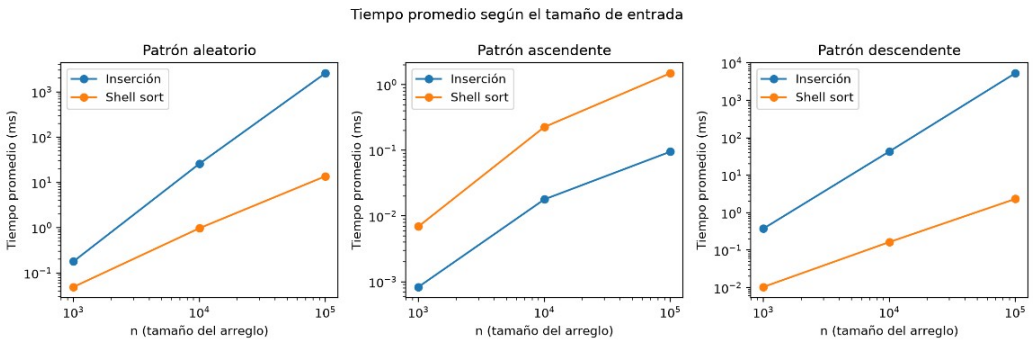


Figura 2: Tiempo promedio de ejecución según el tamaño de entrada (escala logarítmica).

3.2 Comparaciones e intercambios

La Tabla 2 muestra el promedio de comparaciones e intercambios de ambos algoritmos en cada patrón y tamaño, calculado con 30 repeticiones.

Tabla 2: Comparaciones (comp.) e intercambios (interc.), promedio de 30 repeticiones.

n	Patrón	Inserción comp.	Inserción interc.	Shell comp.	Shell interc.
1000	aleatorio	250162	249169	15231	7740
1000	ascendente	999	0	8006	0
1000	descendente	499500	499500	11716	4700
10000	aleatorio	24980062	24970071	264038	149108
10000	ascendente	9999	0	120005	0
10000	descendente	49995000	49995000	172578	62560
100000	aleatorio	2498992318	2498892328	4243409	2793868
100000	ascendente	99999	0	1500006	0
100000	descendente	4999950000	4999950000	2244585	844560

4 Discusión

Con datos descendentes, inserción tuvo que desplazar cada elemento paso a paso a través de todos los anteriores, por eso hizo casi $n^2/2$ comparaciones. Shell sort acorta esos desplazamientos con los saltos grandes, y por eso fue mucho más rápido. En cambio, con datos ascendentes inserción casi no mueve elementos y fue más rápida.

Entre las limitaciones, los tiempos de inserción variaron bastante con n = 10 000 (25.8 ± 6.4 ms), y rand() solo llega hasta 32 767.

5 Conclusiones

1. Con datos aleatorios, Shell sort fue más rápido (13.7 ms contra 2 574 ms de inserción con n = 100 000).

2. Con datos descendentes, Shell sort fue más rápido porque los saltos grandes acortan los desplazamientos (2.3 ms contra 5 266 ms).

3. Con datos ascendentes, inserción fue más rápida porque el arreglo ya está ordenado y casi no mueve elementos (0.095 ms contra 1.5 ms).

4. Al crecer n, la curva de inserción creció mucho más rápido que la de Shell sort con datos aleatorios y descendentes.

5. Una limitación fue que se usó un solo equipo y que no se probaron arreglos casi ordenados ni con duplicados.

Referencias bibliográficas

Salgado, M., Pérez, J., & Albarrán, S. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia e Ingeniería*, *43*(03), 273–280. https://doi.org/10.53766/cei/2022.43.03.05